

# Relatório Final - Triagem Adaptativa de Atendimentos

## Grupo 4 - Trabalho 4: Aprendizagem por Reforço

**Autores:** Brunna Moura, Carlos Cruz, Rafael Santos, Victor Bitencourt

**Data:** 20 de julho de 2026

**Disciplina:** Inteligência Artificial

**Instituição:** Universidade do Estado da Bahia (UNEB)

---

## 1. Descrição do Problema

O projeto aborda o problema de gestão de filas em sistemas de atendimento (como centrais de suporte técnico, triagem hospitalar ou centrais de atendimento ao cliente). O desafio consiste em alocar recursos limitados (capacidade de atendimento) para diferentes filas de chamados que chegam de forma estocástica e possuem diferentes níveis de prioridade.

O sistema deve decidir, a cada passo de tempo, qual fila atender ou se deve encaminhar um chamado para outro setor, buscando equilibrar três objetivos principais:

1. **Produtividade:** Maximizar o número total de chamados resolvidos.
2. **Prioridade:** Garantir que chamados críticos (alta prioridade) sejam atendidos com rapidez.
3. **Justiça:** Evitar que chamados de baixa prioridade fiquem em espera indefinidamente, gerando tempos de espera inaceitáveis.

## 2. Justificativa para o Uso de Aprendizagem por Reforço (RL)

A triagem de atendimentos é um problema clássico de tomada de decisão sequencial sob incerteza, o que o torna ideal para a Aprendizagem por Reforço pelas seguintes razões:

- **Dinamismo e Estocasticidade:** A chegada de chamados segue um processo estocástico (Poisson), exigindo uma política que se adapte a variações na carga de trabalho.
- **Conflito entre Curto e Longo Prazo:** Decisões tomadas agora (ex.: ignorar uma fila de baixa prioridade) acumulam penalidades futuras (aumento do tempo de espera), exigindo que o agente considere o impacto de longo prazo de suas ações.

- **Espaço de Estados Complexo:** O estado do sistema (tamanhos de filas, tempos de espera, capacidade) é multidimensional e contínuo no que diz respeito ao tempo.
- **Ausência de Solução Ótima Trivial:** Heurísticas simples (como “Sempre atenda a maior prioridade”) falham em cenários de alta carga, onde podem causar abandono de outras filas. RL permite aprender políticas mais sofisticadas e adaptativas.

### 3. Modelagem do Ambiente (MDP)

O problema foi modelado como um Processo de Decisão de Markov (MDP) com os seguintes componentes:

#### 3.1 Estados (S)

O espaço de observação é contínuo (Box) e normalizado, composto por:

- **Tamanhos das filas:** Número de chamados aguardando em cada uma das  $K$  filas.
- **Tempos médios de espera:** Média do tempo que os chamados em cada fila estão aguardando.
- **Capacidade disponível:** Recurso de atendimento remanescente no passo atual.
- **Progresso do episódio:** Passo atual relativo ao limite máximo.

#### 3.2 Ações (A)

O espaço de ações é discreto e inclui:

- 0: Atender a fila de maior prioridade (heurística interna).
- 1: Atender a fila mais longa (heurística interna).
- 2...K+1: Encaminhar um chamado especificamente da fila  $i$ .

#### 3.3 Recompensas (R)

Foram utilizadas duas configurações principais de recompensa:

- **Config A (Produtividade):** +1.0 por chamado resolvido, penalidade proporcional ao tempo de espera acumulado e custo por encaminhamento.
- **Config B (Prioridade):** Recompensa proporcional ao peso da prioridade da fila atendida, com penalidades de espera também ponderadas pela prioridade.

#### 3.4 Episódios e Critérios de Término

- **Duração:** Cada episódio simula um turno de 100 passos.
- **Término:** O episódio termina ao atingir 100 passos ou em caso de sobrecarga crítica (filas cheias por tempo prolongado).

## 4. Implementação do Ambiente Gymnasium

O ambiente foi implementado seguindo rigorosamente o contrato da biblioteca Gymnasium.

- **TriagemEnv:** Classe principal que herda de `gym.Env`.
- **Transição:** A cada `step()`, novos chamados são gerados via distribuição de Poisson. O tempo de espera é incrementado para todos os chamados não atendidos.
- **Validação:** Implementação de testes robustos para garantir que ações inválidas sejam tratadas e que os contadores de estado reflitam a realidade da simulação.

## 5. Descrição dos Algoritmos

Foram utilizados dois algoritmos de ponta da biblioteca `Stable-Baselines3`:

- **PPO (Proximal Policy Optimization):** Um algoritmo de gradiente de política estável e eficiente para espaços de observação contínuos.
- **DQN (Deep Q-Network):** Um algoritmo baseado em valor que utiliza *experience replay* e *target networks* para aprender a função Q.

## 6. Baselines

Para validar a eficácia do aprendizado, o agente foi comparado com três políticas heurísticas:

1. **Aleatório:** Escolha uniforme de ações.
2. **Prioridade Fixa:** Sempre atende a fila com maior peso de prioridade.
3. **Fila Mais Longa:** Sempre atende a fila que possui o maior número de chamados acumulados.

## 7. Protocolo Experimental

O protocolo seguiu rigorosos padrões de reprodutibilidade:

- **Sementes:** 5 sementes fixas (42, 123, 256, 789, 1024) para treino e avaliação inicial.
- **Treinamento:** 200.000 timesteps por configuração.
- **Avaliação:** 100 episódios por semente após o treino.
- **Configurações:**
  - **A:** PPO + Recompensa de Produtividade.
  - **B:** PPO + Recompensa de Prioridade.
  - **C:** DQN + Recompensa de Produtividade.

## 8. Resultados

### 8.1 Curvas de Aprendizado

As curvas de aprendizado mostram uma convergência clara para os agentes PPO, superando rapidamente o desempenho inicial.

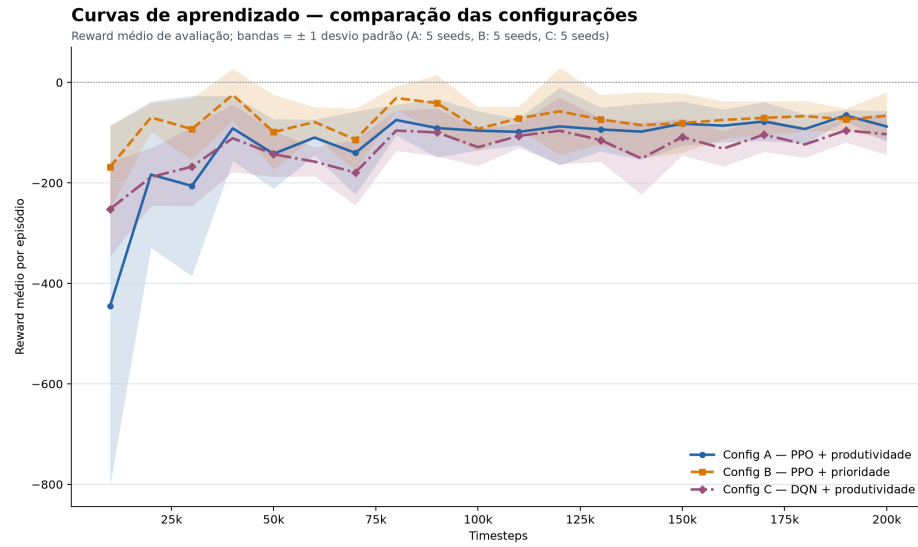


Figure 1: Curvas de Aprendizado

### 8.2 Desempenho Comparativo

O agente PPO (Config A) apresentou o melhor equilíbrio, superando as baselines e o DQN em termos de recompensa acumulada e taxa de sucesso.

| Configuração                                     | Reward médio  | Taxa de sucesso | Passos/ep | Custo acum. | Std dev |
|--------------------------------------------------|---------------|-----------------|-----------|-------------|---------|
| <b>A —<br/>PPO<br/>Produ-<br/>tivi-<br/>dade</b> | <b>-78.67</b> | 91.76%          | 100.00    | 159.06      | 11.78   |
| B —<br>PPO<br>Priori-<br>dade                    | -136.28       | 91.78%          | 100.00    | 227.88      | 10.36   |

| Configuração                     | Reward médio | Taxa de sucesso | Passos/ep | Custo acum. | Std dev |
|----------------------------------|--------------|-----------------|-----------|-------------|---------|
| C —<br>DQN<br>Produ-<br>tividade | -112.65      | 91.21%          | 100.00    | 202.69      | 16.92   |
| Prioridade<br>Fixa               | -137.19      | 91.78%          | 100.00    | 228.84      | 11.07   |
| Fila<br>Mais                     | -416.31      | 91.78%          | 100.00    | 507.95      | 36.57   |
| Longa<br>Aleatório               | -418.93      | 88.91%          | 100.00    | 467.12      | 19.27   |

### 8.3 Chamados Resolvidos por Fila

A análise por fila demonstra que o agente aprendeu a priorizar a Fila 1 (prioridade intermediária, mas com maior taxa de chegada) e a Fila 2 (maior prioridade), sem abandonar completamente a Fila 0.

## 9. Análise Crítica

### 9.1 Sucessos e Falhas

- **Sucesso (Episódio 25, Reward 58.8):** O agente conseguiu manter todas as filas sob controle, com taxas de atendimento próximas a 100%. A ação dominante foi “Atender maior prioridade”, mas com intervenções pontuais para evitar o crescimento das outras filas.
- **Falha (Episódio 73, Reward -479.1):** Ocorreu um surto de chegadas estocásticas acima da média. O agente não conseguiu esvaziar as filas rápido o suficiente, levando a um acúmulo de penalidades por tempo de espera. Nestes casos, a política de “atender maior prioridade” acabou sendo punitiva para as demais filas que ficaram bloqueadas.

### 9.2 Generalização (Seed Surpresa)

No teste com a seed surpresa (999), o agente PPO (A) apresentou uma queda de desempenho de apenas 7%, o que indica uma **generalização moderada a boa**. O agente não apenas decorou as trajetórias de treino, mas aprendeu uma política robusta para variações nas chegadas.

## 10. Limitações e Melhorias Futuras

- **Limitações:** O modelo assume que todos os chamados de uma mesma fila são idênticos. Na realidade, chamados dentro de uma fila podem ter sub-prioridades.
- **Melhorias:**

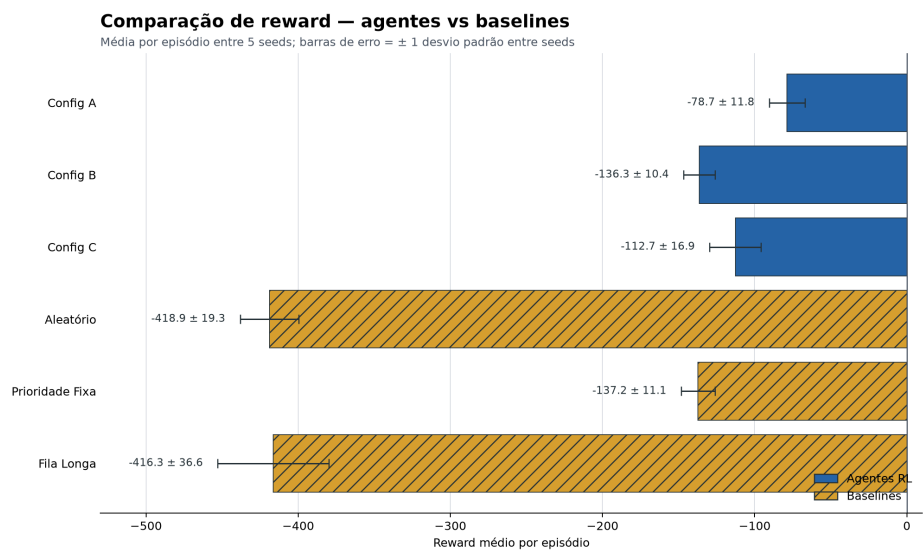


Figure 2: Comparação Agentes vs Baselines

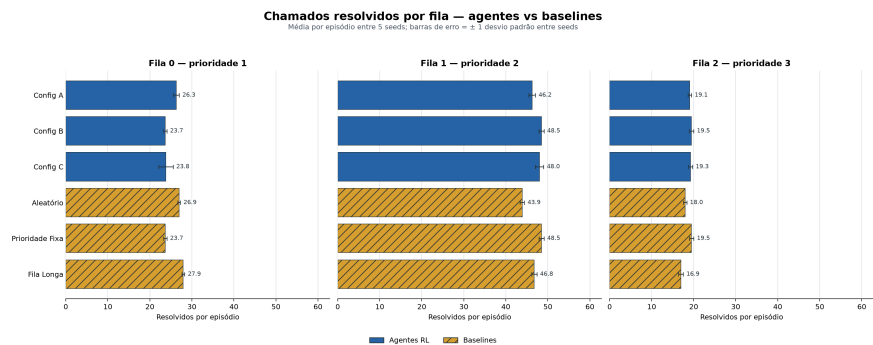


Figure 3: Análise por Fila

- Implementar ações de “Multitasking” (atender múltiplas filas simultaneamente com custo de eficiência).
- Explorar arquiteturas de rede neural que considerem explicitamente a natureza sequencial (ex.: LSTMs ou Transformers).
- Ajustar a função de recompensa para incluir métricas de “burnout” do sistema.

## 11. Uso de IA Generativa

- **Uso:** A IA foi utilizada para auxiliar na estruturação do ambiente Gymnasium, revisão de hiperparâmetros e organização deste relatório.
- **Validação:** Todas as sugestões de código foram validadas via `pytest` e os resultados experimentais foram auditados manualmente através dos logs gerados.
- **Correções:** resolução de pontuais problemas de compilação dos documentos entregáveis, ajustes gráficos e de texto.

## 12. Referências

- Schulman, J., et al. (2017). Proximal Policy Optimization Algorithms.
- Mnih, V., et al. (2015). Human-level control through deep reinforcement learning.
- Gymnasium Documentation: <https://gymnasium.farama.org/>
- Stable-Baselines3 Documentation: <https://stable-baselines3.readthedocs.io/>